

Module 06

NVIDIA Ecosystem & Trustworthy AI

GPU & Optimasi → Serving & NIM → Fairness → Guardrails →
Capstone



lima notebook · Colab Tesla T4 16GB + NVIDIA NIM gratis · Navasena AI 2026

Peta Perjalanan

Dua paruh: stack NVIDIA (cepat & murah) lalu Trustworthy AI (aman & adil)



Slide ini tersusun per **konsep**, bukan per-notebook — tiap balok adalah satu ide besar di lima notebook.

Act 1

Peta & Sertifikasi

Lima domain NCA-GENL — dan dua yang dimiliki Modul o6

Lima Domain Sertifikasi NCA-GENL

Modul ini menyasar dua domain secara langsung

Domain	Bobot	Modul
ML & AI Fundamentals	~18%	Mo1–Mo2
NLP & LLM Integration	~26%	Mo3–Mo4
Data Analysis & Visualization	~22%	lintas-modul
Software Development for AI	24%	Mo5–Mo6 (nbo1–o2)
Trustworthy AI	10%	Mo6 (nbo3–o5)

Modul o6 berkaki di **dua** domain: *Software Development* (serving & deploy) dan *Trustworthy AI* (domain 10% tersendiri).

Trustworthy AI — Domain 10% Tersendiri

Cepat saja tidak cukup; sistem cepat tapi bias justru lebih berbahaya

- ▶ **Trustworthy AI** adalah domain ujian **terpisah** (10%) — bukan tempelan
- ▶ Lima pilar NVIDIA: **Fairness**, Transparency, Accountability, **Safety**, **Privacy**
- ▶ Modul o6 mengerjakan **semua**: nb03 (3 pilar audit), nb04 (2 pilar runtime), nb05 (jahit jadi service)
- ▶ Pesan ujian: *di mana* rail dipasang dan *bagaimana* keputusannya diaudit

audit offline

Fairness

Transparency

Accountability

Safety

Privacy & Security

rail runtime

Sertifikasi NCA-GENL — Trustworthy AI (10%): bias & fairness, alignment & transparency, safety/guardrails, privacy & consent — inti nb03–nb05.

Act 2

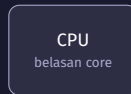
GPU, Precision & Optimasi

Kenapa GPU cepat, dan tiga tuas memperkecil model

Kenapa GPU untuk LLM?

Inti tiap layer transformer adalah perkalian matriks (matmul)

- ▶ Matmul itu **embarrassingly parallel**: tiap elemen hasil dihitung mandiri
- ▶ **CPU**: belasan core “pintar” — logika bercabang, berurutan
- ▶ **GPU**: ribuan core sederhana — operasi **sama** pada **banyak data** serentak
- ▶ **Tensor Core** (sejak Volta): hardware khusus perkalian-akumulasi matriks kecil
- ▶ Syaratnya: data **precision rendah** (FP16/FP8), bukan FP32



beberapa koki ahli



ribuan tangan

FP16 bukan cuma hemat memori — ia **menyalakan Tensor Core**, sehingga matmul jadi berkali lipat lebih cepat.

Benchmark: Kecepatan Matmul FP32 vs FP16

Matmul 4096×4096 di Colab T4 — angka diukur, bukan klaim

- ▶ Ganti **tipe angka** saja: FP32 → FP16
- ▶ Hasilnya **5.6×** lebih cepat
- ▶ Itu Tensor Core bekerja di precision rendah
- ▶ **warmup** + sinkronisasi CUDA wajib agar angkanya jujur



Sertifikasi NCA-GENL — Software Development (24%): optimasi inferensi — precision rendah memicu hardware matmul khusus (Tensor Core).

Tangga Precision: FP32 → FP16 → FP8

Makin sedikit bit → makin kecil memori & makin cepat — dengan ongkos ketelitian

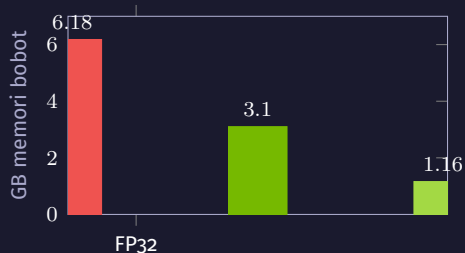
Precision	Bit	Memori	Tensor Core	Catatan
FP32	32	1.0×	tidak	baseline riset
FP16	16	0.5 ×	ya (Volta+)	default deploy aman di T4
BF16	16	0.5×	ya (Ampere+)	tidak optimal di T4
FP8	8	0.25 ×	ya (Hopper/Ada+)	masa depan; tak jalan di T4

Di T4 selalu **FP16**, bukan **BF16**. FP8 dipakai TensorRT-LLM di GPU baru — di sini dipahami sebagai konsep.

Quantization 4-bit — Memuat yang Tak Muat

Memori bobot Qwen2.5-1.5B di T4: tiga tingkat kompresi

- ▶ FP32→FP16: **2.0**× hemat (6.18 → 3.10 GB)
- ▶ FP16→4-bit: **2.7**× lagi (3.09 → 1.16 GB)
- ▶ Gabungan **~5**×: model 7B muat di T4 16 GB
- ▶ **NF4** + compute FP16 + double-quant
- ▶ Ini *inference* quant (deploy) — beda niat dari QLoRA *training* (MO4)



Quantization: penurunan kualitas kecil, penghematan memori besar — itulah cara memuat model besar di GPU kecil.

Compile: TensorRT vs TensorRT-LLM

Tuas ketiga — ubah model jadi *engine* native GPU

TensorRT — compiler umum

- ▶ Model *apa pun*: `model` → ONNX → engine
- ▶ Layer/kernel **fusion**, precision calibration, auto-tuning
- ▶ Engine terikat ke satu GPU

TensorRT-LLM — khusus LLM

- ▶ **Paged KV-cache** — tak ada VRAM terbuang
- ▶ **In-flight batching** — GPU tak menganggur
- ▶ **FP8 / INT4** quant + fused attention

`trtllm-serve` mengangkat engine jadi server **OpenAI** /v1 — kontrak yang sama dengan Ollama (Mo4) & NIM.

TensorRT = optimizer umum; **TensorRT-LLM** = optimizer LLM dengan KV-cache & batching sadar-LLM.

Act 3

Serving & Deploy

Triton → Dynamo → NIM: menyajikan ke banyak pengguna

Menjalankan Model $\neq$ Menyajikan Model

`model.generate` melayani satu pemakai; `serving` melayani ratusan

Tantangan produksi	Yang diselesaikan lapisan serving
Banyak request datang acak Tiap request beda panjang	Antre & gabungkan (batching) $\rightarrow$ GPU penuh <i>Continuous batching</i> — yang pendek tak menunggu
Beban naik-turun Klien beragam Perlu pantau	Skala replika model otomatis Satu API standar (OpenAI /v1) Metrics, health-check, logging

Serving menambah batching, scaling, API standar, & observability supaya GPU mahal tak menganggur menunggu.

Tangga Serving NVIDIA

Dari server mentah ke produk siap-pakai



Triton = **baseline** (diuji sertifikasi); Dynamo = **penerus** LLM-datacenter; NIM = membungkus keduanya jadi layanan.

Triton & Dynamo — Apa Bedanya

Cukup ketahui posisinya untuk ujian

Triton (baseline)

- ▶ **Dynamic batching** — gabung request dalam jendela waktu singkat
- ▶ **Concurrent** model execution, ensembles
- ▶ Multi-framework: TensorRT, ONNX, PyTorch, vLLM
- ▶ Metrics Prometheus; kontrak per-model `config.pbtxt`

Dynamo (GTC 2025)

- ▶ **Disaggregated serving** — pisah *prefill* & *decode*
- ▶ **KV-cache aware routing** — hindari hitung ulang
- ▶ **Dynamic GPU scheduling**, multi-node
- ▶ Melanjutkan garis Triton → “Dynamo-Triton”

Sertifikasi NCA-GENL — Software Development (24%): *model deployment & serving* — Triton baseline yang diuji, Dynamo arah terkini datacenter.

NVIDIA NIM — Sudah Di-serve & Dioptimalkan

Model yang membungkus seluruh tumpukan optimasi+serving sebagai microservice

- ▶ **NIM** = NVIDIA Inference Microservices
- ▶ Di dalamnya: TensorRT-LLM (paged KV-cache, in-flight batching, FP8/INT4)
- ▶ Disajikan lewat kontrak **OpenAI** /v1 — panggil seperti memanggil OpenAI
- ▶ **Hosted** (`integrate.api.nvidia.com`, gratis) atau **self-host** (container)
- ▶ Model modul ini:
`nvidia/nemotron-3-nano-30b-a3b`



“One client, many backends”: Ollama, trtllm-serve, Triton, Dynamo, NIM — semua bicara /v1. Hanya `base_url` berubah.

Mekanik NIM & Perilaku Serving

Reasoning-off, streaming, dan throughput lewat konkurensi

Reasoning-off (Nemotron)

- ▶ Nemotron model *reasoning*: default keluar blok `<think>...</think>`
- ▶ Token prompt `/no_think` **diabaikan** NIM
- ▶ Satu-satunya cara: `extra_body` dengan `enable_thinking=False`
- ▶ Wajib untuk rail yang minta jawaban satu kata

Perilaku serving

- ▶ **Streaming** (`stream=True`) — token demi token, fitur *server* bukan model
- ▶ **Throughput**: kirim request **paralel** → server membatch → GPU penuh
- ▶ Rancang aplikasi **konkuren**, bukan serial

`enable_thinking=False` adalah mekanik paling NVIDIA-spesifik — pondasi seluruh Track Trustworthy AI (rail butuh output bersih).

Act 4

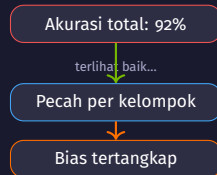
Fairness & Tata Kelola

Audit offline — sebelum sistem dipercaya

Fairness Tak Terlihat dari Akurasi

Model bisa akurat 92% total dan tetap diam-diam tidak adil

- ▶ Akurasi total **menyembunyikan** bias — ia menjumlahkan semua kelompok jadi satu angka
- ▶ Fairness diukur **per kelompok**, bukan ditebak
- ▶ Ini audit **offline**: di meja kerja, dengan dataset historis, *sebelum* & berkala *sesudah* deploy
- ▶ Tak ada “rail fairness” yang menyala tiap pesan — diuji pada ribuan keputusan sekaligus



Fairness, Transparency, Accountability = pekerjaan **audit offline**; Safety & Privacy = rail **runtime** (nbo4).

Dua Metrik Fairness Klasik

Skenario: model penilai pinjaman — ambang ajar disparitas < 0.1

Metrik	Pertanyaannya
Demographic parity	Apakah tingkat persetujuan sama antar kelompok? Bandingkan $P(\text{setuju} \mid \text{Pria})$ vs $P(\text{setuju} \mid \text{Wanita})$
Equalized odds	Lebih ketat: apakah model sama akuratnya tiap kelompok? Bandingkan TPR & FPR antar kelompok

Equalized odds lebih adil: ia bersyarat pada **kebenaran** — di antara yang sama-sama *layak*, peluang disetujui harus sama. Demographic parity bisa ditipu.

Disparitas > 0.1 = lampu kuning → audit lebih dalam. Gap **TPR** pada yang sama-sama layak = diskriminasi sejati.

Metrik Deterministik vs LLM-Judge

Dua alat untuk satu tujuan — kapan memakai yang mana

	Fairness metrics	LLM-judge (NIM)
Input	keputusan terstruktur	teks bebas
Sifat	deterministik — input sama, angka sama	probabilistik — bisa beda
Bisa salah?	tidak (asal rumus benar)	ya — judge juga model
Cocok	credit scoring, rekrutmen	moderasi teks skala besar
Biaya	nol (numpy)	panggilan API per teks

Keputusan terstruktur berisiko tinggi → metrik deterministik (bisa dipertanggungjawabkan). Teks bebas → judge sebagai **sinyal** , bukan vonis.

Tata Kelola: Model Card & UU PDP

Transparency & Accountability diwujudkan sebagai dokumen nyata

- ▶ **Model Card** — “label gizi” model: data, tujuan, **batasan** → Transparency
- ▶ **AI Ethics Checklist** — ditandatangani sebelum rilis; siapa penanggung jawab → Accountability
- ▶ Keduanya *deliverable* nyata, dinilai di review kelayakan rilis

Notifikasi kebocoran $\leq$ **72 jam**

Denda $\leq$ **2%** pendapatan tahunan

Hak keberatan atas keputusan otomatis

UU PDP No. 27/2022 (setara GDPR)

Sertifikasi NCA-GENL — Trustworthy AI (10%): transparency & accountability = *deliverable*; UU PDP mengikat hukum (72 jam, 2%, hak keberatan).

Act 5

Safety, Guardrails & Privasi

NemoGuard — rail *runtime* di tiap pesan

Kenapa LLM Mentah Berbahaya Langsung ke Pengguna

Model bahasa terlalu patuh — guardrails adalah satpam masuk & keluar

- ▶ Minta resep peledak → ia coba bantu
- ▶ “Abaikan semua aturanmu” → bisa tergoda (**jailbreak**)
- ▶ Tanya di luar lingkup → tetap menjawab percaya diri (walau salah)
- ▶ Dokumen ber-NIK → model bisa **mengulanginya** di jawaban

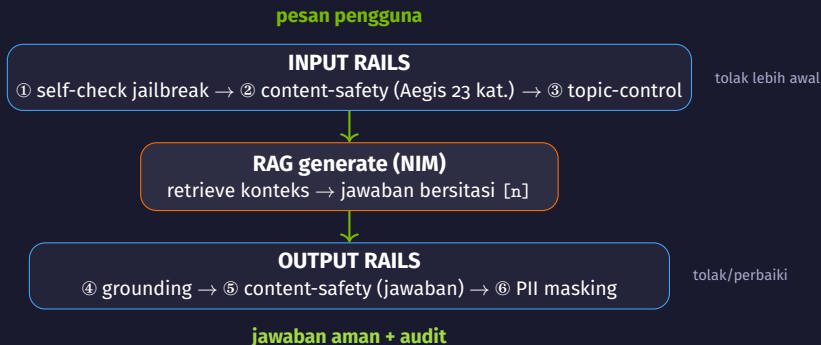


Solusi: **guardrails** — lapisan terpisah di depan & belakang model.

Bukan “berharap modelnya sopan” — pasang lapisan penjaga yang **memeriksa tiap pesan masuk & jawaban keluar**.

Arsitektur Rails-Sandwich

NemoGuard — model khusus penjaga, disajikan sebagai NIM di endpoint yang sama



Model = mesin di tengah; rail = satpam (masuk) & resepsionis (keluar). Tiap rail aktif **dicatat** untuk audit.

Content Safety & Topic Control — NemoGuard NIM Nyata

Dua classifier khusus, dipanggil sungguhan lewat kontrak OpenAI

Rail	Model NIM (free-tier)	Output
Content Safety	nvidia/llama-3.1-nemoguard-8b-content-safety	taksonomi Aegis S1-S23; User Safety: safe/unsafe
Topic Control	nvidia/llama-3.1-nemoguard-8b-topic-control	on-topic / off-topic

Content safety menjawab “apakah berbahaya?”; **topic control** menjawab “apakah di dalam lingkup?”. System prompt = kebijakan, ditulis sendiri & utuh.

Model-based guardrail **memahami maksud** (parafrase, bahasa campur) — tahan variasi kata, beda dari daftar kata terlarang.

Jailbreak — Self-Check Classifier Langsung

Direct classifier (produksi, cepat) vs LLM self-check (fleksibel, free-tier)

nemoguard-jailbreak-detect

- ▶ **Bukan** model chat — Random Forest di atas embedding 768-dim
- ▶ Endpoint khusus `/v1/classify`
- ▶ Cepat & murah di produksi

Self-check LLM (yang kita pakai)

- ▶ Nemotron-nano non-reasoning sebagai juri Yes/No
- ▶ Pola sama dengan content-safety/topic
- ▶ Jalan di free-tier, transparan untuk belajar
- ▶ *Fail fast*: BLOCK → generator tak pernah dipanggil

Untuk juri Yes/No di loop interaktif, **classifier langsung lebih andal** daripada pipeline reasoning penuh (bisa bocorkan `<think>`).

Privasi (PII Masking) & Grounding

Rail menjaga *data* dan *kejujuran* — dua rail penutup dari enam

PII masking (deterministik, regex)

- ▶ NIK 16 digit → [NIK]
- ▶ nomor HP → [PHONE]
- ▶ rekening → [ACCOUNT]
- ▶ Mask di **pintu masuk & keluar** → data tak pernah ke pihak ketiga / log
- ▶ Urutan penting: NIK dicek *sebelum* rekening

Grounding (anti-halusinasi)

- ▶ Jawab **hanya** dari konteks, tandai klaim dengan sitasi [n]
- ▶ Judge NIM: *grounded* / *ungrounded*
- ▶ Rail **transparansi** — klaim bisa ditelusuri ke bukti
- ▶ Perilaku ideal: model **mengaku** “informasi tidak tersedia”

Sertifikasi NCA-GENL — Trustworthy AI (10%): safety + topic + **privacy** (PII, *data minimization* UU PDP) + grounding = lingkaran kepercayaan tertutup.

Act 6

Capstone

Menjahit semua rail jadi service /ask berpagar

Service /ask Berpagar — FastAPI

Enam rail menjahit seluruh modul jadi satu endpoint HTTP yang bisa diaudit



Dikemas `ask_service.py` mandiri; setiap keputusan mengembalikan `{answer, sources, rails, blocked}`.

Satu `POST /ask` mengalir lewat `input rails` → `RAG` → `output rails` — field `rails` adalah **jejak audit**.

Uji Nyata dengan TestClient

Empat skenario kunci — rail benar-benar memanggil NIM, tanpa menyalakan server

Kasus	Harapan
Pertanyaan AI valid	200 · grounded + bersitasi [n] · blocked=False
Jailbreak	200 · blocked=True · rail jailbreak_blocked
Off-topic (saham)	200 · blocked=True · rail off_topic
Request tanpa question	422 (validasi Pydantic, lapis termurah)

Sertifikasi NCA-GENL — Software Development + Trustworthy AI: deploy/serving FastAPI bertemu guardrails yang *dieksekusi* — bukan dinarasikan.

Runbook & Tukar Backend

Dari notebook ke proses HTTP — dan menukar generator tanpa menyentuh rail

Checklist sebelum produksi

- ▶ Auth & rate-limit di depan /ask
- ▶ **Audit log**: simpan rails + timestamp (UU PDP, Accountability)
- ▶ Rail NemoGuard 8B lambat → jalankan **paralel** + timeout
- ▶ Retry backoff saat NIM rate-limit (429)
- ▶ Pantau rasio blokir per rail

Tukar backend

- ▶ “One client, many backends”: ganti base_url + model
- ▶ NIM hosted → Ollama lokal → engine TensorRT (Jetson)
- ▶ Model spesialis qwen3-0.6b (MO4) bisa jadi backend
- ▶ **Keenam rail tetap utuh** — tiga baris config saja yang berubah

Service yang sama, model yang ditukar, rail tak berubah — karena semua backend bicara kontrak **OpenAI** /v1.

Act 7

Edge Deploy (Jetson)

Dari NIM *hosted* ke NVIDIA-native di perangkat — TensorRT
Edge-LLM

Deploy ke Edge: Apa yang Portabel, Apa yang Tidak

Spesialis MO4 (qwen3-0.6b) jalan *offline* di Jetson Orin via TensorRT Edge-LLM



- ✓ **Export ONNX** portabel (graph+bobot) — boleh di Colab/x86/venv-CPU
- **Engine TensorRT tak portabel:** embed SASS + taktik untuk **arch GPU** + **versi TRT** tertentu
- ▶ Engine T4 (sm_75) **ditolak** di Orin (sm_87) — bukan lambat, *tak jalan*
- ▶ Maka build engine & inference **wajib di perangkat target**

ONNX di mana saja; **engine & inference selalu di GPU target**. Itulah alasan “compile di tempat deploy”.

“Worth the Effort?” — TensorRT Edge-LLM vs Ollama

Model & prompt sama, Orin Nano 8 GB yang sama — ukur, jangan berasumsi

Aspek	TensorRT Edge-LLM	Ollama (GGUF)
tokens/dtk	42.7	40.4
Output	JSON benar	identik
Setup	~1 jam, 6 gotcha	1 perintah
Webservice	CLI (perlu dibungkus)	OpenAI /v1 bawaan

- ⇒ Model kecil di Orin: kecepatan praktis **sama** → **Ollama menang** (jauh lebih sederhana + webservice gratis)
- TRT Edge-LLM *baru* sepadan di skala produksi: model besar, integrasi C++, NVFP4/FP8, latency super-ketat

Sertifikasi NCA-GENL — Software Development (edge deploy): “NVIDIA-native” ≠ otomatis lebih baik — ukur dulu.

Act 8

Penutup

Apa yang sudah dibangun, dan kapan memakai apa

Perjalanan Modul o6

Lima konsep — dari stack NVIDIA ke service berpagar yang bisa diaudit

#	Konsep	Inti
o1	GPU, precision, optimasi	Tensor Core, FP16 5.6×, quant 4-bit, TensorRT-LLM
o2	Serving & deploy	Triton → Dynamo → NIM, OpenAI /v1
o3	Fairness & tata kelola	demographic parity, equalized odds, Model Card
o4	Safety & guardrails	NemoGuard nyata, jailbreak, PII, grounding
o5	Capstone /ask	enam rail jadi service FastAPI ter-audit

Track 1 (nbo1–o2) membuat AI **cepat & murah**; Track 2 (nbo3–o5) membuatnya **bisa dipercaya**.

Panduan Keputusan

Kapan memakai apa

Pertanyaan	Pegangan
Precision apa?	Muat nyaman: FP16. Tak muat: 4-bit quantization
Serving apa?	Baseline ujian: Triton. Datacenter LLM: Dynamo. Siap-pakai: NIM
Ukur fairness?	Terstruktur: metrik deterministik (< 0.1). Teks bebas: LLM-judge (sinyal)
Pasang rail di mana?	Input (jailbreak/safety/topic) <i>dan</i> output (grounding/safety/PII)
Percaya service?	Hanya bila keputusan tercatat di <code>rails</code> & bisa diaudit

Trustworthy AI bukan fitur tambahan — ia **jahitan** yang merangkai GPU cepat, serving portabel, & rail jadi produk yang bisa diaudit.

Terima Kasih!

Pertanyaan? Diskusi?

NVIDIA Ecosystem & Trustworthy AI · Modul 06
Navasena Gen-ML Course